

Migrating OpenPiton's Native Coherence Protocol to AMBA CHI

A phased RTL design proposal for the OpenPiton manycore platform

Prepared by Nameer Iqbal Ansari

RTL Design Engineer (RISC-V SoC / Cache-Coherence)

Independent proposal, submitted for review and collaboration



Proprietary and Confidential

Table of Contents

1. Executive Summary	3
2. Motivation and Benefits	3
2.1 Why change a working protocol	3
2.2. Concrete Benefits	3
2.3. What this proposal is not claiming	3
3. Current Architecture (Baseline)	4
4. Identified Gaps In The Current Implementation	4
5. Related Work And Prior Art	4
6. Proposed Architecture Phase 0 (Base Protocol Conversion)	5
6.1. Node Mapping	5
6.2. Tile-level connectivity under CHI: RN-F, HN-F, and SN-F wiring	7
6.3. Channel Mapping	7
6.3.1 Field-Level Packet Mapping	8
6.3.2 NoC Implementation Strategy: Physical Networks vs. Virtual Channels	9
6.4. Conflict Resolution: Retry And Protocol Credit	10
6.4.1 RN-F Replay / Credit Machinery In Detail	11
6.5. State Machine And Memory-Model Preservation	12
6.5.1 What TSO Requires	12
6.5.2 Reconstructing The Same Guarantees on CHI	12
6.5.3 A Related Simplification: Retiring ReqWBGuard	14
6.5.4 Assumptions This TSO Argument Depends On	14
6.6 Non-Coherent Sideband for IPIs and Diagnostics	15
7. RTL Change Inventory (Phase 0)	15
7.1. Additional MicroArchitectural Implications	17
8. Explicitly Out of Scope For Phase 0	18
Phase 1: Richer Snoop Types, Exclusive Atomics, And Block-Store / Prefetch Reactivation	18
Phase 2: Directory Scaling And CDR Integration	19
Phase 3: DVM Broadcast Operations And Multi-Chip CHI Export	19
9. Broader Impact On The OpenPiton Project	19
10. Anticipated Reviewer Questions	20
11. References	21



1. Executive Summary

OpenPiton is a tiled, scalable, open-source manycore platform from Princeton University's Wentzlaff Parallel Research Group, built around a directory-based MESI coherence protocol implemented across three custom on-chip networks (NoC1–NoC3). This protocol has served the platform well since its 2015 tape-out, but it is bespoke: its packet formats, conflict handling, and node roles exist only inside OpenPiton, which limits who can interoperate with it and how rigorously its edge cases can be reasoned about.

This proposal describes converting OpenPiton's coherence layer to AMBA CHI (Coherent Hub Interface), the industry-standard directory-based coherence protocol used across ARM-based and increasingly RISC-V-based multi-core and chiplet systems. The conversion is scoped as a phased program. This document specifies Phase 0, the base protocol conversion, in full RTL-level detail, and outlines the shape of later phases without committing their design yet.

This document is intended to be a single, self-contained reference: the gaps in the current implementation, the benefits of the change, the exact RTL surfaces affected, what is deliberately deferred, and the broader project-level impact, so a reviewer can evaluate the proposal without needing outside context.

2. Motivation and Benefits

2.1 Why change a working protocol

OpenPiton's coherence protocol is functionally complete for its original goal as a research vehicle built around a modified OpenSPARC T1 core, but its scope is narrower than what the platform has grown into. OpenPiton now hosts multiple cores through community extensions (see Section 7), and multi-core, multi-vendor coherence work increasingly assumes a standard protocol at the boundary. A bespoke protocol is a barrier to that.

2.2. Concrete Benefits

- **Interoperability:** any CHI-compliant request node (RN), not just cores ported specifically to OpenPiton's CCX/L1.5 interface, becomes a candidate for integration.
- **Formal conflict handling:** CHI's retry/protocol-credit mechanism and home-node order point give a standard, well-understood answer to races that OpenPiton's spec currently leaves undocumented (Section 4).
- **Ecosystem reuse:** CHI's message classes, node roles, and channel structure are documented independently of OpenPiton, so design decisions can be checked against an external, versioned specification rather than only against OpenPiton's own source.
- **Positions OpenPiton for chiplet-era research:** standard coherence fabrics are the connective tissue of UCIe- and CXL-adjacent multi-die work; a CHI-speaking OpenPiton is directly relevant there.
- **Directory design headroom:** CHI's node-ID and system address map concepts give a cleaner path to scaling past OpenPiton's current 64-sharer directory ceiling (Section 8).

2.3. What this proposal is not claiming



Proprietary and Confidential

This is not a claim that OpenPiton's native protocol is broken for its current uses; it is a working, taped-out, well-tested design. The case here is that a CHI-based coherence layer extends what OpenPiton can be used for and closes specific, identifiable gaps, at a cost this document details rather than hides.

3. Current Architecture (Baseline)

OpenPiton's coherence system has three participants: the private, per-tile L1.5 write-back cache (the requester), the distributed, shared, inclusive L2 cache (the point of coherence, with directory state co-located per line), and the off-chip memory controller. Coherence messages move across three physical, priority-ordered NoCs: NoC1 carries L1.5 → L2 requests, NoC2 carries L2 → L1.5 forwards/downgrades and L2 → memory requests, and NoC3 carries acknowledgements and writebacks and is never blocked. This strict priority ordering (NoC3 > NoC2 > NoC1) plus point-to-point ordering per NoC is what currently gives the protocol its deadlock freedom.

MESI state per cache line is tracked in the L2's co-located state array and directory array; the L1.5 is inclusive of the OpenSPARC T1's L1D and transduces the CCX bus protocol to NoC packets. Full detail is in the OpenPiton Microarchitecture Specification [1] and the original ASPLOS paper [2].

4. Identified Gaps In The Current Implementation

These are gaps that are either stated in OpenPiton's own specification or have been noted by engineers who have studied the protocol independently. Each is cited to its source rather than asserted.

- Coherence transaction conflicts are undocumented. The specification's state-transition diagrams (its Figures 5–10 in the OpenPiton `micro_arch` doc) show clean, non-conflicting transitions only. An independent technical review of cache coherence protocols observed that OpenPiton's specification "does not say anything about how they deal with coherence transaction conflicts," unlike, for example, AMBA CHI's own documentation, which devotes a chapter to interconnect protocol flows and race handling [3].
- The directory's sharer vector is fixed at 64 bits, hard-limiting precise sharer tracking to 64 cores per L2 slice, as stated directly in the specification's L2 overview [1, Sec. 5.1].
- Several packet-level features are present in the format but functionally disabled: prefetch requests are treated as no-ops, and block-store requests are decomposed into regular stores before reaching the L2, both stated explicitly in the specification's coherence-operations section [1, Sec. 3.1].
- Coherence Domain Restriction (CDR), the mechanism for interleaving cache lines across a software-specified subset of L2 tiles, is described in the specification itself as "an experimental research feature" [1, Sec. 2.3.3], not a hardened part of the coherence design.
- The conflict-resolution gap and the fixed sharer-vector width both become load-bearing the moment OpenPiton is expected to interoperate with external, standard-protocol IP, which is the direct motivation for this proposal.

5. Related Work And Prior Art

This proposal does not start from an empty field. Two active efforts are directly adjacent and are referenced throughout this document:



Proprietary and Confidential

- **BYOC / BlackParrot-BedRock:** a pluggable, heterogeneous cache-coherence framework a pluggable, heterogeneous cache-coherence framework for tiled multicores structurally analogous to OpenPiton, allowing different cores and coherence systems to be composed together [4]. It demonstrates that OpenPiton's L1.5/CCX boundary is already a proven point of substitution, the same boundary this proposal's RN-F transducer targets.
- **EPAC ("The Last Dance"):** a tiled manycore design whose uncore is built natively on AMBA CHI, with a distributed L2 cache slice co-located with a CHI home node architecturally close to what Phase 0 of this proposal produces for OpenPiton [5].
- **Coherence Domain Restriction (CDR):** Fu, Nguyen, and Wentzlaff's MICRO-48 paper on interleaving cache lines across a software-specified L2 tile subset [6], the mechanism referenced as experimental in Section 4 above and revisited in Section 8's Phase 2 scope.
- **Positioning:** this proposal is narrower than EPAC (it retrofits an existing, non-CHI platform rather than designing CHI-native from scratch) and adjacent to BYOC (both target the same substitution boundary, but BYOC's goal is pluggable heterogeneity while this proposal's goal is standard-protocol compliance).

6. Proposed Architecture Phase 0 (Base Protocol Conversion)

Phase 0 is scoped to be a complete, functionally closed conversion: every coherence transaction OpenPiton supports today must still work after Phase 0, expressed in CHI's channels, node roles, and packet formats, including a standard answer to the conflict cases Section 4 identifies. Phase 0 deliberately does not add new coherence functionality beyond what CHI requires to express OpenPiton's existing behavior; richer features are Section 8's job.

6.1. Node Mapping

CHI defines three node roles relevant here: Request Nodes (RN) issue transactions, Home Nodes (HN) are the point of coherence and ordering for an address, and Subordinate Nodes (SN) back memory. OpenPiton's three coherence participants map onto these directly, because the L2 already combines the roles of shared cache and co-located directory that CHI's HN-F formalizes:

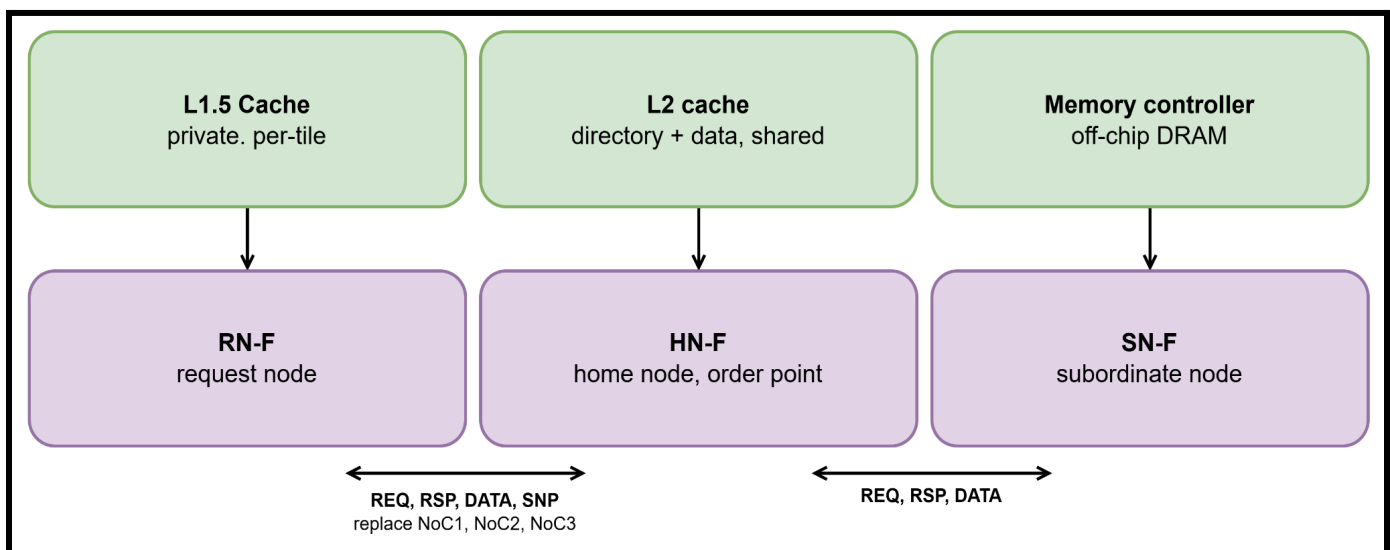


Figure 1: OpenPiton component to CHI node-role mapping, and channel replacement for the three native NoCs



Proprietary and Confidential

OpenPiton Component	CHI Role	Rationale
L1.5 cache (private, per-tile)	RN-F (fully coherent request node)	Already the sole per-tile coherence requester owns MESI state for its lines.
L2 cache + co-located directory	HN-F (fully coherent home node)	Already the point of coherence and the only place directory state lives.
Memory controller	SN-F (fully coherent subordinate node)	Already the sole backing store behind the L2; no protocol role changes.

Table 1: CHI Node assignment

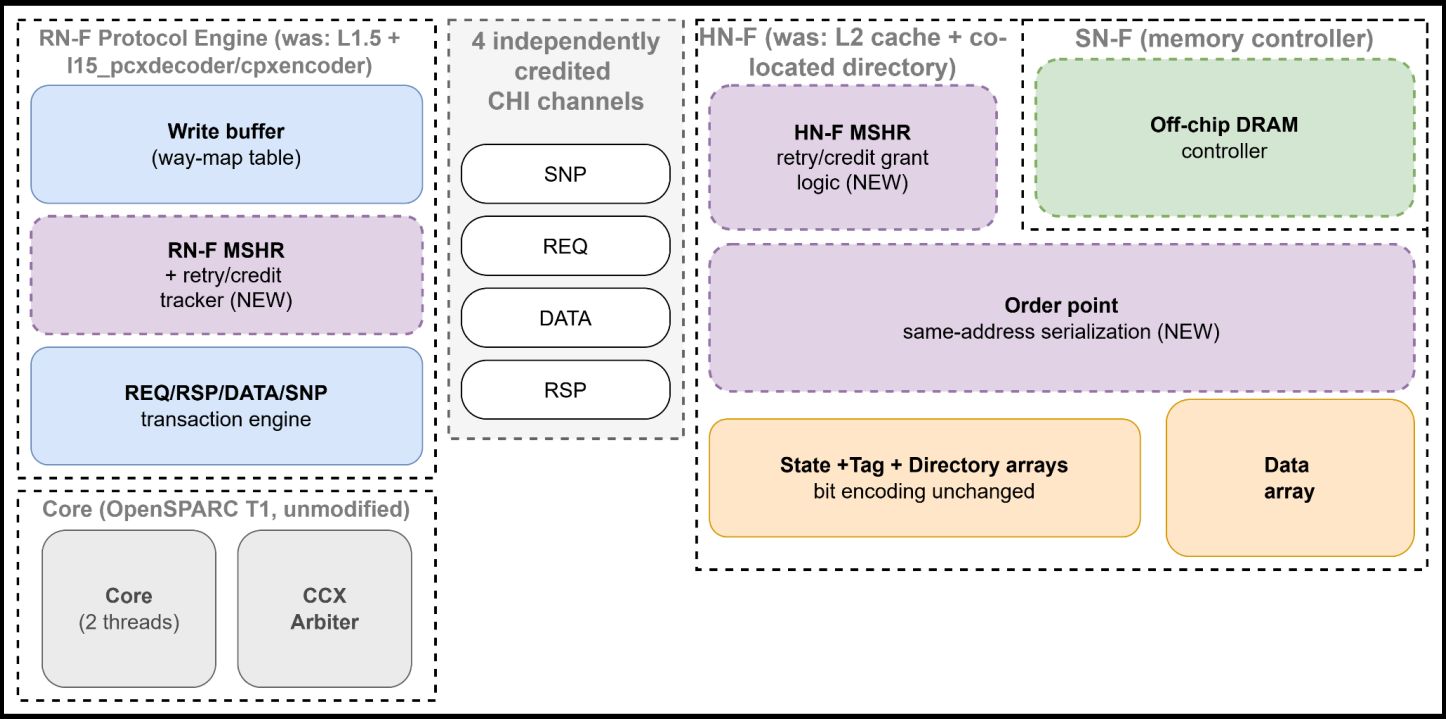


Figure 2: Overview of updated tile microarchitecture

The L1.5 becomes an RN-F protocol engine, the L2 becomes an HN-F with an explicit order point, and the three priority-ordered NoCs become four independently-credited CHI channels. Dashed purple blocks are new logic introduced by Phase 0; solid blocks are existing structures whose bit-level content is preserved and whose decode logic is retargeted



6.2. Tile-level connectivity under CHI: RN-F, HN-F, and SN-F wiring

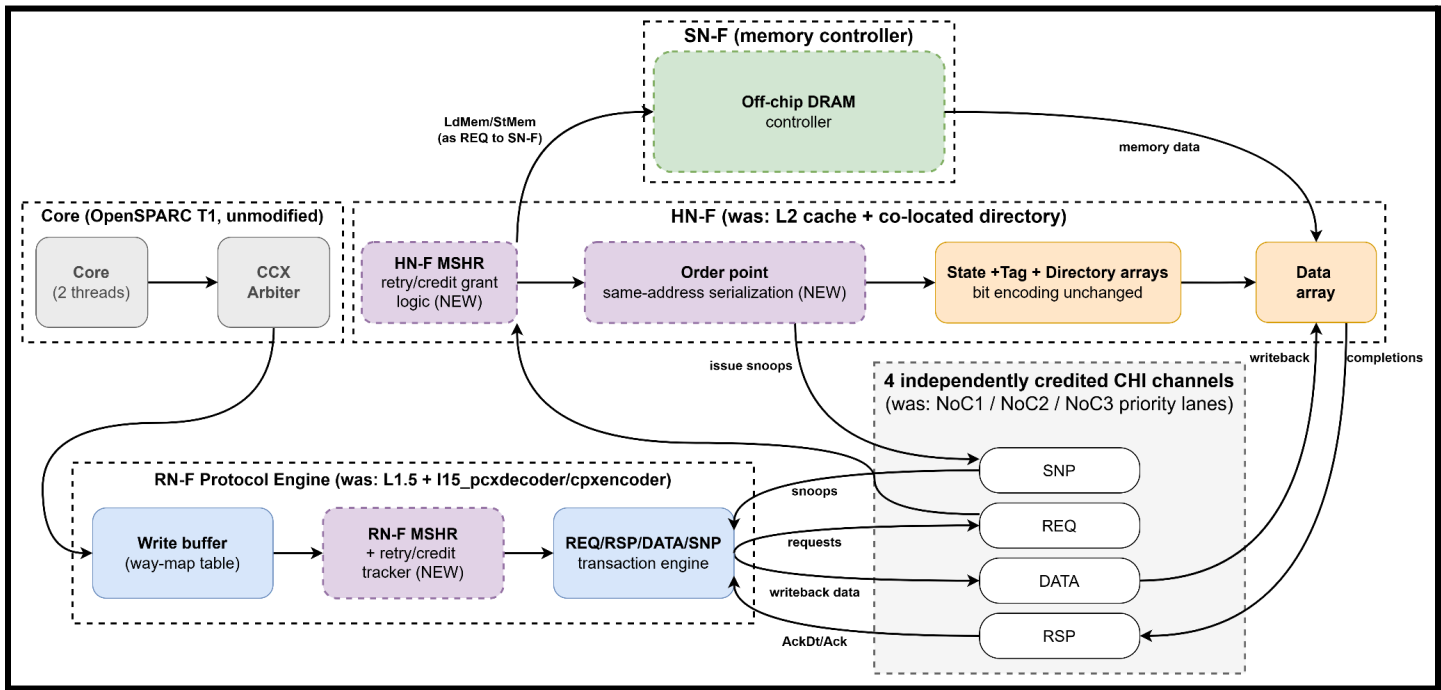


Figure 3: Connectivity flow of tile microarchitecture

This diagram is the RTL-level payoff of the whole node-mapping argument in Section 6.1: it shows the same three OpenPiton components (L1.5, L2, memory controller) redrawn as their CHI roles, with the new logic Phase 0 has to add called out explicitly (the purple/amber blocks retry/credit tracker, MSHR retry-state, order point, credit-grant logic) sitting alongside the structures that carry over unchanged (blue/orange write buffer, state/tag/directory arrays, data array). The four channels in the gray box on the right are the direct replacement for NoC1/NoC2/NoC3: instead of three priority-ordered physical lanes carrying mixed traffic, each of SNP/REQ/DAT/RSP carries exactly one traffic class end-to-end, which is what makes the retry/credit and ordering arguments in Sections 6.3–6.4 provable rather than incidental.

6.3. Channel Mapping

CHI separates traffic into four logical channels: REQ (requests), RSP (control-only responses), DAT (data-bearing responses), and SNP (snoops from HN to RN), each with independent credit-based flow control. OpenPiton's three NoCs mix data and control traffic by priority tier rather than by type. Phase 0 replaces the NoC1/NoC2/NoC3 priority scheme with CHI's four-channel scheme:

Native NoC	Native traffic	CHI channel(s)
NoC1 (L1.5 → L2)	ReqRd, ReqExRd, ReqAtomic	REQ
NoC2 (L2 → L1.5 / memory)	FwdRd, FwdExRd, Inv, LdMem, StMem, Ack, AckDt	SNP (to RN-F), REQ (to SN-F), RSP/DAT (acks/data)



Native NoC	Native traffic	CHI channel(s)
NoC3 (L1.5/memory → L2)	FwdRdAck, FwdExRdAck, InvAck, ReqWB, LdMemAck, StMemAck	RSP (control acks), DAT (writeback data)

Table 2: CHI channel mapping

This is not a relabeling exercise. Today's NoC1 priority level carries both address-only requests and, in the block-store path, inline data; CHI requires these to travel on physically distinct channels with independent credit pools. That separation is the source of most of the RTL change inventory in Section 7.

6.3.1 Field-Level Packet Mapping

Section 6.2 establishes which CHI channel each class of native traffic moves to; it does not by itself show that the fields inside OpenPiton's packet headers have somewhere to go. This subsection carries the mapping down to the bit level, using the OpenPiton specification's own header tables (its Tables 1–8) as the source. This is the level of detail needed to judge Phase 0's feasibility rather than just its intent.

Native Field (Spec Source)	Width/Role	CHI Equivalent	Mapping Notes
CHIPID, XPOS, YPOS (Table 1, FLIT1)	14+8+8 b - destination tile	Target Node ID (TgtID)	The (chipid, x, y) tuple already uniquely names a tile; Phase 0 flattens it into a single CHI node ID via the same address-to-tile lookup the router uses today.
SRC CHIPID, SRC X, SRC Y (Table 1, FLIT3)	14+8+8 b - requester tile	Source Node ID (SrcID)	Carried in every CHI REQ/SNP/DAT/RSP flit; a direct, CHI-native equivalent of the same information.
FBITS / SRC FBITS (Table 1, Table 5)	4 b - router egress port	Absorbed into the interconnect's node-ID-to-port routing table	CHI resolves SrcID/TgtID to a physical port at the network layer, below the protocol layer; Phase 0's routers keep an FBITS-equivalent lookup internally, but it is no longer a field carried on the wire.
MSHR/TAG (Table 1, Table 2)	8 b	Transaction ID (TxnID)	One-to-one behavioral equivalents both exist solely so the requester can match a response to its outstanding request.
MESSAGE TYPE (Tables 1–4, Table 6)	8 b, ~36 defined values	Opcode (separate REQOpcode / RSPOpcode / DATOpcode / SNPOpcode)	OpenPiton's single flat message-type space splits across CHI's four independent per-channel opcode fields, matching Section 6.2's channel split.
PAYLOAD LENGTH (Table 1, Table 3)	8 b - flit count	Implicit in fixed-beat DAT bursts + DataID	CHI's data channel has no explicit length field; each CompData/WriteData flit instead carries a DataID marking its position in a burst sized by the transaction's Size field.
OPTIONS2 sub-cache-line bit vector (Table 7)	4 b - dirty/present 16 B sub-lines within a 64 B line	Byte Enable (BE) on WriteData/CompData	CHI's BE is byte-granular, finer than OpenPiton's 16 B granularity; Phase 0 sets whole-beat BE bits per dirty 16 B segment, a narrowing, not a loss, of precision.



Native Field (Spec Source)	Width/Role	CHI Equivalent	Mapping Notes
OPTIONS2 icache bit (Table 7)	1 b	No direct CHI field; kept as RN-F/HN-F-local routing metadata	CHI has no protocol-level I\$/D\$ distinction; OpenPiton already resolves this at the L2 today (spec Sec. 2.4), so Phase 0 keeps it off the wire and inside HN-F request decode.
OPTIONS2 data-size (Table 8)	3 b, 0-64 B	Size field on ReadNoSnp/WriteNoSnp	Direct equivalent; CHI's non-snooping-transaction Size field already encodes the same range used for non-cacheable and atomic operations.
OPTIONS4 fill-cacheline-state (Table 7)	2 b (M/E/S)	Resp field on CompData (e.g. UC, SC, UD, SD)	Direct equivalent; CHI's cache-state encoding in Resp is a superset of OpenPiton's 3-state fill indication.
OPTIONS4 L2-miss bit (Table 7)	1 b	No CHI protocol field; retained as HN-F-internal telemetry	Plausibly related to the L2 access/miss counters described in Sec. 5.4.9–10; irrelevant to RN-F correctness, so Phase 0 keeps the L2 access/miss counters (Sec. 5.4.9–10) as HN-F-local registers instead of putting the bit on the wire.
OPTIONS4 last-sub-cacheline bit (Table 7)	1 b	Final-beat indication via DataID + burst length	Direct equivalent; CHI already knows the last beat of a burst once DataID reaches the position implied by the transaction's size.

Table 3: Native packet fields to CHI field mapping

Two rows above the L2-miss bit and the icache bit are the only fields Phase 0 cannot place directly on the CHI wire. Both are OpenPiton-internal signals with no bearing on inter-node protocol correctness, so retiring them from the packet format (while preserving their function locally) is a deliberate simplification, not a gap.

This mapping is also what the updated tile microarchitecture below implements: the same header-decode logic that today reads OpenPiton's native fields (l15_pcxdecoder.v / l15_cpxencoder.v, Section 7) becomes the RN-F/HN-F logic that reads and writes the CHI-equivalent fields in the table above.

6.3.2 NoC Implementation Strategy: Physical Networks vs. Virtual Channels

Section 6.3's four CHI channels are a protocol-layer concept; CHI itself does not mandate how they are carried on-chip. This is a real implementation decision Phase 0 has to make, and it deserves its own discussion because OpenPiton's existing choice at this layer is unusually explicit: the specification states its three NoCs “are physical networks (no virtual channels)” [1, Ch. 6], each with two dedicated 64-bit unidirectional links, and derives its deadlock-freedom argument directly from that physical separation plus the NoC3 > NoC2 > NoC1 priority ordering (Section 3, Section 6.3 of this proposal).

Two options exist for carrying CHI's four channels:

- **Physical separation (recommended for Phase 0):** four dedicated unidirectional link pairs, one per channel, replacing today's three. This preserves OpenPiton's existing deadlock-avoidance argument almost unchanged, since each channel keeps its own buffering and cannot be blocked by traffic on another channel, the same property NoC3's “never blocked” guarantee relies on today. The cost is wiring and buffer area: four physical lanes instead of three, at the router, chip bridge, and inter-tile mesh level. The on-chip area cost of the fourth physical lane (increased router area and mesh wire congestion at every hop, not only at the chip bridge) is accepted as a Phase 0 cost in exchange for



reusing the existing deadlock argument; virtual-channel multiplexing (below) remains available as a later area-recovery step once Phase 0 is stable, but is not a Phase 0 substitute for this cost.

- **Virtual channels over shared physical links:** multiplexing REQ/RSP/DAT/SNP as virtual channels over fewer physical links, which is how many production CHI implementations reduce wiring area. This reintroduces exactly the coupling problem OpenPiton's physical separation was designed to avoid: two virtual channels sharing a physical link need their own escape-channel or bubble-reservation discipline to remain deadlock-free, independent of the message-level PCrdGrant credit scheme in Section 6.3. That is a second, distinct deadlock argument layered underneath the protocol-level one, and it is not free to get right.

Phase 0 recommends the physical-separation option. It is the lower-risk path precisely because it lets Phase 0 reuse OpenPiton's existing, already-verified deadlock argument almost as-is, changing only the channel count from three to four; the RTL change inventory in Section 7 is scoped on this assumption. Virtual-channel multiplexing is a legitimate later area optimization, but it is deferred out of Phase 0 deliberately, since it would require re-deriving deadlock freedom at the link layer as a second, independent proof, on top of the protocol-layer argument Section 6.4 already has to make. This is flagged explicitly so a reviewer does not assume Phase 0 is silent on it, or that the four-channel design is free of area cost.

6.4. Conflict Resolution: Retry And Protocol Credit

This is the direct answer to the gap in Section 4. CHI resolves resource conflicts at the home node with a standard two-part mechanism: if the HN-F cannot accept a request (for example, its miss-status holding register, MSHR, pool is full), it returns a RetryAck instead of silently stalling the channel; the requester holds the transaction without re-issuing; when a resource frees, the HN-F sends an explicit protocol credit grant (PCrdGrant); only then does the requester re-send, and the HN-F's order point serializes the retried request against anything else in flight for that address.

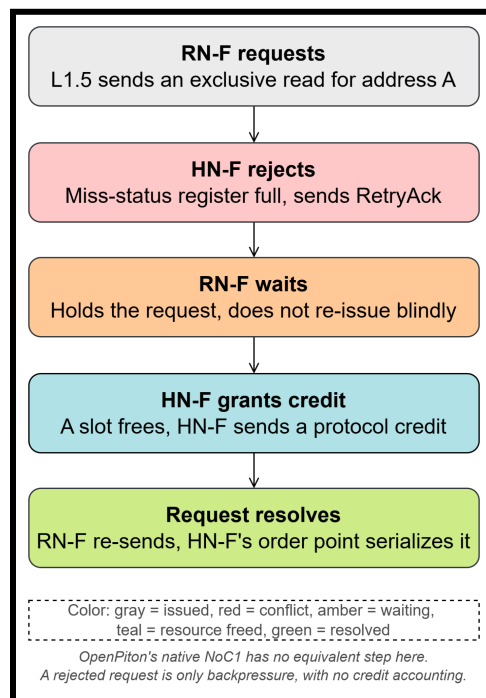


Figure 4: CHI's retry/credit conflict-resolution sequence, replacing OpenPiton's undocumented backpressure-only behavior



Under the native protocol, the equivalent situation of an MSHR-full condition at the L2 is handled by stalling the requesting NoC1 lane (Section 5.3's Stage 1 MSHR CAM check), with no explicit protocol-level accounting of the rejected transaction. Phase 0 replaces this with the CHI sequence above, which is what closes the conflict-handling gap rather than papering over it.

6.4.1 RN-F Replay / Credit Machinery In Detail

Figure 4 shows the sequence at the level a reviewer needs to judge the protocol argument; it does not show what the RN-F actually has to hold in flip-flops to make that sequence work. This subsection specifies that machinery.

- Sizing this buffer is not a general goal but a hard requirement: replay-buffer capacity at every RN-F must be provisioned to be at least six entries, the maximum number of outstanding transactions the SPARC/PCX interface can generate (one read, one write, one icache request, per thread, across two threads; specification Section 4.8.1) so that a fully retried RN-F can never be forced to stall PCX backpressure for want of replay-buffer space. This closes the circular-dependency hazard where the core cannot retire a PCX transaction pending a CHI credit while the RN-F simultaneously has no room to park the retried request; sizing the buffer to the core's maximum outstanding count removes that dependency by construction rather than by verification alone.
- Backpressure under credit starvation. The six-entry sizing above bounds the number of concurrent transactions the core-side PCX interface can generate; it does not bound how long any one of them may sit parked awaiting PCrdGrant. Under sustained HN-F or fabric congestion, grants can be delayed arbitrarily, and a fixed-depth replay buffer alone cannot absorb an unbounded wait. Phase 0 therefore requires the RN-F to assert PCX-level backpressure stalling new request acceptance from the core whenever replay-buffer occupancy reaches its six-entry capacity, rather than accepting new PCX requests it has no room to park. This converts credit starvation into a bounded core stall (safe, and already a normal PCX behavior per Section 4.10) rather than a buffer overflow. This backpressure path, together with the arrival-order grant-arbiter fairness argument in Section 6.3.1 (which guarantees every retried request is eventually granted, not merely ordered), is what keeps credit starvation from becoming unbounded rather than merely bounding the number of concurrent retries.
- TxnID is preserved across replay. When the RN-F re-issues after PCrdGrant, it reuses the original TxnID rather than allocating a new one. This matters for the HN-F's order point (Section 6.4.2): reusing the TxnID lets the HN-F recognize a replayed request as the same transaction it previously rejected, rather than a new arrival, which keeps the arrival-order argument in Figure 4 well-defined even across a retry.
- Credit accounting is per RN-F, not global. PCrdGrant is a bilateral credit between one RN-F/HN-F pair, not a shared pool across all requesters. This is what prevents a single busy address from starving unrelated traffic: an RN-F waiting on a credit for address X can still have other, unrelated transactions to address Y in flight and accepted, since those consume separate MSHR/credit slots at the HN-F.
- Grant arbitration avoids livelock. When multiple RN-Fs are simultaneously retried against the same HN-F resource class, the HN-F's credit-grant arbiter (a new block, alongside the order point in Figure 2,3) grants PCrdGrant in strict arrival order of the original RetryAck'd request, not round-robin or priority order. This guarantees every retried request eventually receives a grant in bounded time, which is the liveness property Section 6.4's TSO argument silently depends on: an order-point argument is meaningless if a request can be retried indefinitely.
- Interaction with the credit/retry tracker in the L1.5 pipeline. The tracker introduced in Figure 6 (Section 7.1) is the RN-F-side counterpart of this machinery: it is the block that actually holds the parked transaction, counts outstanding PCrdGrant credits, and re-drives S1 on grant. Sections 6.3 and 6.3.1 describe the protocol-level behavior this tracker must implement; Section 7.1 gives its RTL placement.



Proprietary and Confidential

6.5. State Machine And Memory-Model Preservation

Phase 0 keeps OpenPiton's MESI state encoding in the L2 state array unchanged at the bit level; what changes is which message types drive each transition. The existing $I \rightarrow S$, $I \rightarrow E$, $I \rightarrow M$, and eviction diagrams (the specification's Figures 5–10) are re-expressed using CHI's request and snoop message names, but the state graph itself, including the silent $E \rightarrow M$ transition, is preserved exactly, since Phase 0's mandate is behavioral equivalence, not a new coherence model.

CHI itself is memory-model agnostic; it does not mandate TSO. OpenPiton's TSO guarantee today comes from NoC point-to-point ordering combined with the OpenSPARC T1 core's own ordering behavior; the specification attributes it explicitly to "point-to-point ordering" (Sec. 2.4), not to the $\text{NoC3} > \text{NoC2} > \text{NoC1}$ priority tiers, which exist for deadlock avoidance. That distinction matters: it means TSO was never actually resting on the priority scheme Phase 0 removes, but the argument needs to be made explicitly rather than assumed, which is the purpose of this subsection.

6.5.1 What TSO Requires

For this discussion, three properties of TSO matter: (a) each core observes its own loads and stores in program order; (b) all cores agree on a single total order of stores to any given address; (c) a load returns the value of the most recent store to that address in this order (or the core's own more recent, not-yet-globally-visible store). OpenPiton's native mechanism supplies these through three properties working together: the T1 core issues PCX requests to the L1.5 in program order; NoC point-to-point ordering preserves that order all the way to the L2; and the L2 processes requests to a given cache line strictly serially (single MSHR/state-array access per line, enforced by index-locking), which is the actual single order point behind property (b).

6.5.2 Reconstructing The Same Guarantees on CHI

Phase 0 rebuilds properties (a)–(c) from CHI's own ordering primitives, none of which are new invariants; each has a direct counterpart in what OpenPiton already relies on:

- Program order into the fabric (property a): unchanged from today. The core-facing CCX/PCX pipeline stays in-order per thread; Phase 0 does not touch this path, so the RN-F still issues requests to the HN-F in the order the core generated them.
- Per-pair point-to-point ordering: CHI implementations are permitted to guarantee ordering for transactions between a given RN-F/HN-F pair on a given channel, exactly the property OpenPiton's NoC1 already guarantees today. Phase 0's RN-F/HN-F link is designed to this ordered profile, so a given L1.5's requests still arrive at the HN-F in issue order.
- A single order point per address (property b): the HN-F's MSHR/state-array access remains serialized per cache line, the same index-locking mechanism as today (Section 5.3's Stage 1 MSHR CAM check), just repurposed to gate CHI transaction acceptance instead of native message types. Because every RN-F's request for a given address funnels through this one serialization point, the HN-F still produces a single global order for same-address operations, regardless of which RN-F they came from.
- Snoop-response-before-completion (property c): CHI requires an RN-F to return its SnpResp (and, where applicable, write back or forward its data) before the HN-F issues completion (Comp/CompData) to the new requester. This is the same thing OpenPiton's $\text{Inv} \rightarrow \text{InvAck} \rightarrow \text{AckDt}$ sequence already enforces (spec Section 3.2's $I \rightarrow M$ diagrams): a downgrade must finish before the new owner's data becomes visible. Phase 0 keeps this rule as a hard requirement on the HN-F's completion logic, not an optional optimization.



Proprietary and Confidential

- Data-visibility gating on completion, not on data arrival (property c, continued). CHI permits CompData and its ordering completion to be decoupled: a requester can receive data over the DAT channel before or independently of the transaction's ordering-completion signal. If the RN-F made a load's data visible to the core as soon as CompData arrived, rather than waiting for the HN-F's ordering point to have actually serialized the transaction, a core could observe a value before the HN-F has globally fixed that transaction's position in per-address order reintroducing exactly the kind of stale-visibility hazard TSO forbids, without violating any CHI-level rule. Phase 0 therefore adds an explicit requirement on the RN-F: data received via CompData is buffered but not forwarded to the core (and not used to satisfy a subsequent local load-store forward) until the HN-F's completion signal for that transaction has been received. This is new RN-F-side gating logic, not present in the native L1.5 today (which has no separate data/completion split to gate), and is called out explicitly here so it is not mistaken for something CHI enforces automatically.

Composed together, these four properties reproduce the same visibility order TSO requires: each RN-F's own requests stay in order (a, matching property a); the HN-F gives every address a single global serialization (b, matching property b); and that serialization is made visible to all observers in a consistent sequence because completions cannot outrun the snoop responses that establish ownership (matching property c). None of this depends on OpenPiton's specific NoC3 > NoC2 > NoC1 priority scheme consistent with the specification's own attribution of TSO to ordering rather than priority (Sec. 2.4).

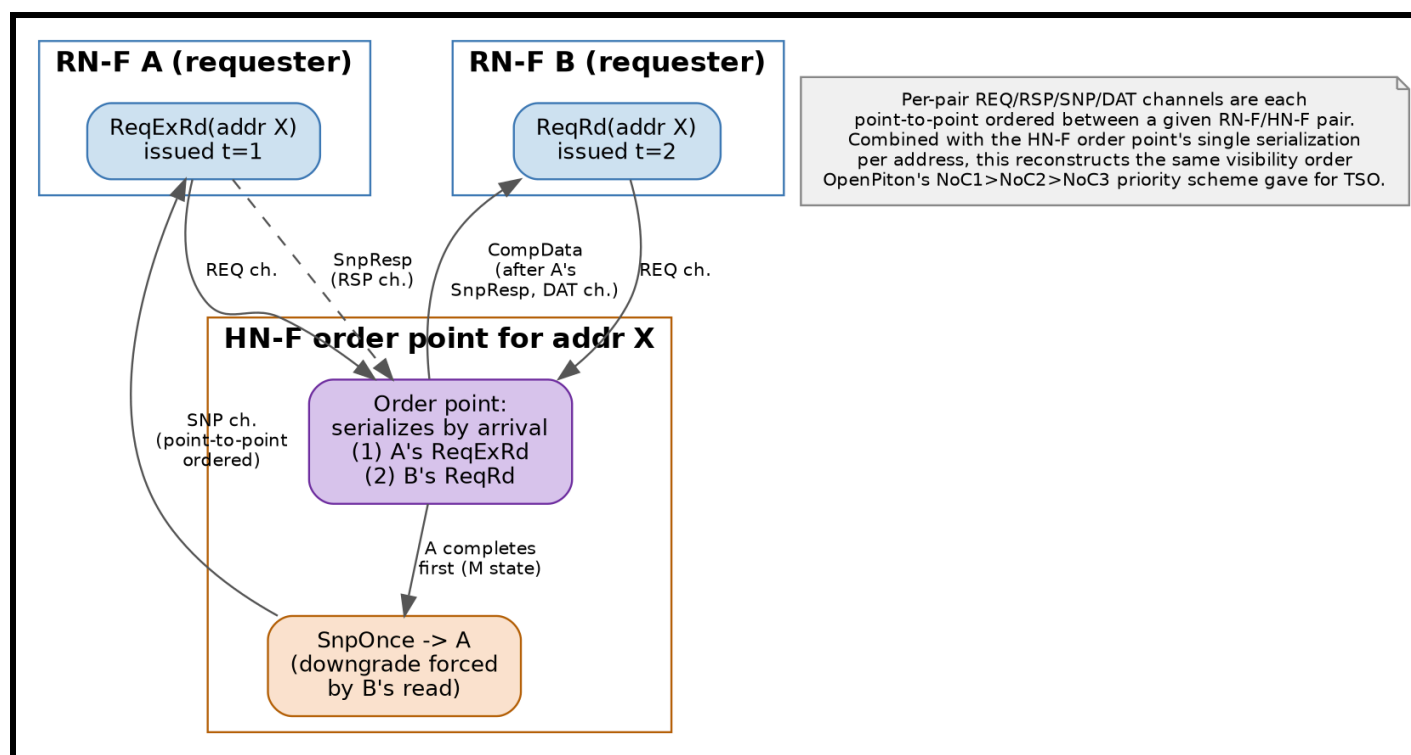


Figure 5: TSO reconstruction on CHI

The HN-F order point serializes two RN-Fs' requests to the same address, and per-pair channel ordering plus the snoop-response-before-completion rule make that serialization visible in a consistent order to both requesters, the same guarantee OpenPiton's NoC point-to-point ordering plus per-line index-locking give today



6.5.3 A Related Simplification: Retiring ReqWBGuard

One concrete consequence of having an explicit HN-F order point is worth flagging on its own, since it is a case where Phase 0 is not a 1:1 translation. OpenPiton's ReqWBGuard message (spec Section 3.2, Table 6) exists to stop a later read or exclusive-read request from overtaking an in-flight write-back and reaching the directory first a race that has to be handled explicitly because NoC1 and NoC3 are physically separate lanes with independent point-to-point ordering. On CHI, a WriteBack (or WriteUnique) and a subsequent Read to the same address both land in the HN-F's single per-address order point described above, so the ordering that ReqWBGuard exists to enforce is already guaranteed structurally. Phase 0 therefore retires the WBGuard message class entirely rather than giving it a CHI counterpart; this is called out explicitly in the RTL change inventory (Section 7.1), so it isn't mistaken for an oversight.

6.5.4 Assumptions This TSO Argument Depends On

Section 6.5.2's argument is only as strong as the implementation choices it depends on, and CHI leaves several of those choices open rather than mandating them. Listing them explicitly is what lets a reviewer check the argument rather than take it on faith:

- Ordered, not relaxed, per-pair channels. CHI permits an implementation to relax point-to-point ordering between an RN-F/HN-F pair for area or performance reasons; property (a)'s counterpart above requires Phase 0 to specifically choose the ordered profile for the REQ and DAT channels on this link. This is a design decision this proposal is making, not a guarantee CHI hands over for free, and it should be stated as a requirement in any implementation spec that follows this document.
- One active transaction per RN-F per address at a time. The order-point argument (property b) assumes an RN-F does not have two outstanding requests to the same address simultaneously; OpenPiton's native L1.5 already enforces this today via its per-thread, one-outstanding-request-per-address discipline (spec Section 4.5.3), and Phase 0's RN-F must preserve that discipline rather than relax it for the sake of exploiting CHI's higher available concurrency.
- Snoop-response-before-completion is enforced as a hard rule, not a best-effort one. Some CHI profiles allow speculative or early completion in exchange for a more complex recovery protocol; property (c) requires Phase 0's HN-F to never take that option. This is explicitly a conservative implementation choice, made in favor of TSO-preservation over the additional performance some CHI profiles could otherwise offer.
- The credit-grant arbiter is fair and starvation-free (Section 6.3.1). The order-point argument is a safety property (same-address requests get a consistent order); it says nothing about liveness (that a retried request eventually gets served). That liveness guarantee rests entirely on the arrival-order grant policy in Section 6.3.1, which is a Phase 0 design choice, not an automatic CHI property.
- Scope: this argument covers Phase 0's transaction set only. Section 8's later phases (richer snoop types, exclusive atomics, DVM) introduce message classes not analyzed here; each should be checked against these same four properties before being folded into the TSO argument, rather than assumed compatible by extension.
- RN-F gates data visibility on completion, not on data arrival. CHI's DAT and RSP channels are independently credited and may deliver CompData and completion out of step with each other; property (c)'s counterpart above requires Phase 0's RN-F to withhold data from the core-side pipeline and from local store-to-load forwarding until completion is received for that transaction, even though CHI permits presenting the data earlier. This is a Phase 0 design commitment, not a default RN-F behavior, and should be listed alongside the other four assumptions as part of the TSO claim's actual scope.



Proprietary and Confidential

Because CHI itself provides no ordering guarantee as a protocol-level safety net, these five assumptions are enforced entirely by Phase 0's RTL, not by CHI compliance checking. A narrow race in the protocol translation layer, for example, a snoop arriving at the RN-F concurrently with a store-buffer flush to the same address, can violate TSO while remaining fully CHI-compliant, since CHI has no visibility into OpenPiton's memory-consistency contract. Directed simulation against the existing regression suite is necessary but not sufficient to catch this class of bug. Formal verification of the four ordering properties in Section 6.5.2 via model checking (e.g., JasperGold) or a Mur ϕ -style protocol model of the RN-F/HN-F boundary is a Phase 0 tape-out gating requirement, not a stretch goal, and should be scoped and resourced alongside the RTL work in Section 7 rather than treated as post-implementation verification overhead.

None of these assumptions are unusual for a CHI-based design; they are the same choices real CHI implementations make when a strict memory model is required. The point of listing them here is that Phase 0's TSO claim (Section 10's second question) is conditional on all five holding, not on CHI's base specification alone, and a reviewer should treat this list as the actual scope of the claim.

6.6 Non-Coherent Sideband for IPIs and Diagnostics

Phase 0 preserves a dedicated non-coherent path alongside the four CHI channels specifically for IPI delivery and L2 special-address diagnostic/configuration accesses (specification Sections 4.7 and 5.4). This path is implemented as phase 0 implements this sideband as a reserved, non-coherent opcode range on the RSP channel, rather than as a fifth physical lane. RSP is chosen because it is control-only (no data payload conflict with DAT) and already carries acknowledgment-class traffic with independent credit accounting (Section 6.3); IPI and diagnostic-register packets are distinguished from ordinary CHI RSP traffic by a dedicated opcode subrange, decoded by the RN-F/HN-F as non-coherent before entering the normal completion path. This keeps Phase 0's physical channel count at four (Section 6.3.2) rather than five, avoiding an additional physical-lane cost on top of the REQ/RSP/DAT/SNP expansion, and is explicitly out of CHI's protocol scope; it is a transition-period mechanism, retired in Phase 3 once DVM-based IPI delivery (Section 8) is in place. Phase 0's RTL change inventory (Section 7) and regression validation (Section 9) both assume this sideband exists from the start of Phase 0, not as a Phase 1+ addition.

Ordering against coherent traffic. Because sideband packets travel outside the REQ/DAT completion path that Section 6.5's TSO argument governs, they require their own explicit ordering rule relative to coherent CHI transactions from the same RN-F. Phase 0 requires the RN-F to preserve program order between sideband requests and coherent requests it issues to the same HN-F: a sideband request (MMIO, IPI, diagnostic access) is not issued until all older coherent requests from that RN-F have received their completion, and a coherent request is not issued until all older sideband requests have been acknowledged. This mirrors the ordering guarantee OpenPiton's native NoC1 point-to-point delivery already provides between mixed request types today (spec Section 2.4), and is necessary because the sideband's reserved-opcode mechanism (above) does not by itself participate in the HN-F's per-address order point the way REQ/DAT traffic does. Without this rule, an MMIO write could complete out of order relative to a cacheable store from the same core, which is exactly the hazard SPARC/RISC-V fence semantics are designed to prevent.

7. RTL Change Inventory (Phase 0)

This section lists the concrete RTL surfaces Phase 0 touches, using the module and structure names from the current specification and source tree, so scope is unambiguous.

- **I15_pcxdecoder.v / I15_cpxencoder.v:** Currently translates OpenSPARC T1's CCX bus signals to and from OpenPiton's internal NoC1/NoC3 encodings. These become the RN-F protocol engine: encoding L1.5 requests as CHI REQ transactions and decoding SNP/RSP/DAT traffic back to CCX responses.



Proprietary and Confidential

- **L1.5 pipeline (stages S1–S3, specification Section 4.10):** Today's S1 stall logic (index-locking, address conflict, full MSHR) is extended with an explicit RetryAck/credit-wait state per Section 6.3, replacing implicit backpressure with tracked protocol credits. Figure 6 below shows the extended pipeline.
- **L2 MSHR metadata array (specification Table 13):** Extended with CHI transaction-ID and retry-state fields alongside the existing address, way, MSHR-ID, and message-type fields.
- **L2 state array and directory array (specification Section 5.2.2, Table 12):** bit encoding unchanged in Phase 0; the decode logic that reads and writes these arrays is updated to key off CHI message types rather than OpenPiton's native message-type table (specification Table 6).
- **Packet header definitions (specification Tables 1–8):** Redefined as CHI-compliant fields per the field-level mapping in Section 6.2.1: node IDs replace CHIPID/XPOS/YPOS, a CHI transaction ID replaces MSHR/TAG, and the message-type table is replaced by CHI's opcode set per channel.
- **dynamic_node_top.v, dynamic_input_control.v, dynamic_output_control.v, space_avail_top.v (on-chip network, specification Chapter 6):** The crossbar and buffer-management logic is extended from three priority-ordered lanes to four independently credited CHI channels; the existing credit-based flow control concept is retained but re-scoped per channel instead of per priority tier.

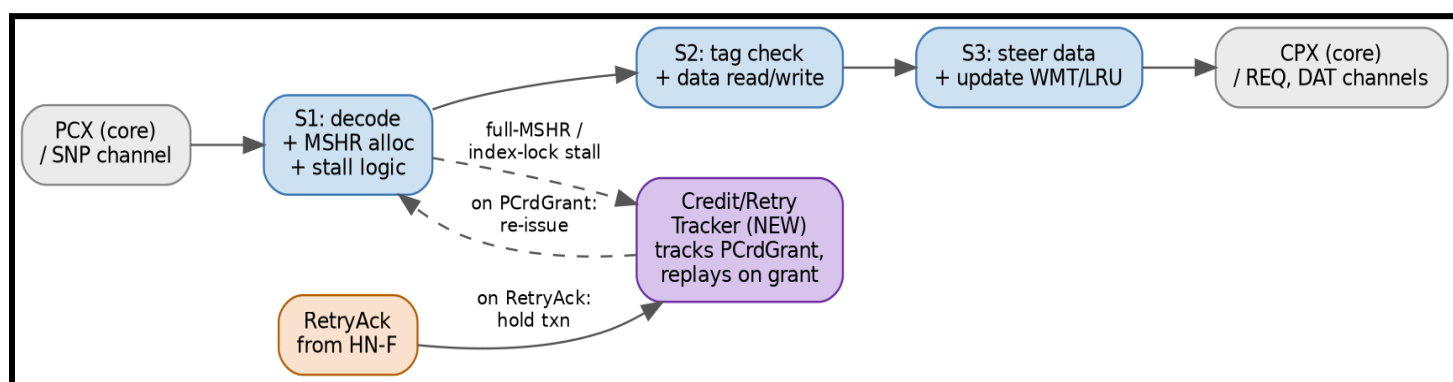


Figure 6: L1.5 pipeline (specification Figure 11) extended with a credit/retry tracker

S1's existing stall conditions (index-lock, address conflict, full MSHR) now also gate on outstanding PCrdGrant state, and a RetryAck from the HN-F holds the transaction for replay instead of being silently absorbed by NoC backpressure.

- **Chip bridge (specification Section 2.5.2):** Currently multiplexes three NoCs over a single narrower off-chip link; extended to multiplex four CHI channels using the same asynchronous-buffer structure.
- **Configuration/build system:** A build-time toggle (native protocol vs. CHI protocol) is required so existing configurations and test infrastructure keep working during the transition (Section 9). To avoid merging native and CHI decode logic into shared conditional paths inside timing-sensitive structures l15_pcxdecoder.v/l15_cpxencoder.v, the L1.5 S1–S3 pipeline, and the L2 MSHR metadata array and its decode logic (all listed above), Phase 0 keeps native-mode and CHI-mode logic in **physically separate module instances selected at build time**, rather than as inline conditionals within a single shared module. This avoids adding conditional-logic depth to the native critical path (which must not regress) as the cost of introducing the CHI path, at the expense of some code duplication between the two module variants.



- **Formal TSO verification model:** A model-checked or Mur ϕ -style formal proof that the four properties in Section 6.5.2 hold at the RN-F/HN-F boundary across all Phase 0 transaction types, gating Phase 0 sign-off per Section 6.5.4.
- **RN-F data/completion gating logic:** Buffers CompData and withholds core-visible forwarding until the matching completion signal arrives for that transaction, per Section 6.5.2's data-visibility requirement.
- **RN-F sideband/coherent ordering logic:** enforces program-order sequencing between sideband (RSP-opcode) requests and coherent REQ/DAT requests issued by the same RN-F, per Section 6.6.

7.1. Additional MicroArchitectural Implications

The bullets above are the RTL surfaces Phase 0 directly rewrites. A few more implications fall out of the conversion without being changed to a named module in their own right; a reviewer estimating effort should account for these too.

- ReqWBGuard retirement. As Section 6.5.3 describes, the write-back-guard message class is dropped rather than translated, because the HN-F order point already provides the ordering it existed to enforce. This removes one message type and its associated NoC1-insertion logic rather than adding one.
- The way-map table (WMT) is untouched. The L1.5's way-map table (specification Section 4.3), which reconciles independent way allocation between L1D and L1.5, is a purely local structure that CHI has no visibility into or opinion about. Phase 0 leaves it exactly as-is; this is worth stating explicitly so a reviewer does not assume the CHI conversion touches core-facing cache-management structures it does not.
- PCX queue depth vs. CHI credit accounting. The core can have up to 6 concurrent outstanding requests today (1 read + 1 write + 1 icache request per thread, 2 threads specification Section 4.8.1), buffered by a 2-entry PCX queue at the L1.5. The new RN-F credit/retry tracker (Section 6.3, Section 7) has to account for this existing core-side concurrency on top of CHI's own PCrdGrant bookkeeping, so PCX-side backpressure and CHI-side protocol-credit exhaustion do not deadlock against one another. This is guaranteed structurally, not just verified in simulation, by the six-entry replay-buffer sizing rule specified in Section 6.4.1. This is an interaction Phase 0 must verify even though neither side, in isolation, is new.
- Non-cacheable and diagnostic accesses move to CHI's non-snooping requests. Non-cacheable loads/stores (specification Section 4.5.1) and the L2's special diagnostic and register accesses (Section 5.4.1–5.4.11) map to CHI's ReadNoSnp/WriteNoSnp request types. These CHI request types do not, by definition, invalidate other RN-Fs' copies automatically, so the existing two-transaction behavior (L1.5 first flushes/invalidates its own and L1D's copy, then forwards the request, per Section 4.5.1) must be preserved explicitly as an RN-F-local sequence ahead of issuing the ReadNoSnp/WriteNoSnp, rather than assumed to come for free from the transaction type.
- Non-coherent sideband is in Phase 0 scope, not deferred. Inter-processor interrupts (specification Section 4.7) and the L2's special diagnostic/configuration-register accesses (specification Section 5.4) currently ride NoC1/NoC2 alongside coherence traffic. Because Phase 0 retires NoC1/NoC2 as general-purpose transports, a non-CHI hardware sideband or a vendor-defined use of one CHI channel reserved for non-coherent control packets is a required Phase 0 deliverable, not a Phase 3 cleanup item. Without it, the chip cannot complete interrupt delivery or diagnostic register access during Phase 0 bring-up, which blocks full-system Linux boot and prevents Phase 0 from being validated against the existing regression suite (Section 9). This sideband is retired only in Phase 3, once CHI's DVM messages take over IPI delivery (Section 8).
- Chip bridge bandwidth is redistributed, not just re-muxed. Today's chip bridge splits a fixed 2.8 GB/s (at the 350 MHz target frequency, specification Section 2.5.2) three ways across NoC1–3. Going to four independently-credited



channels changes the per-channel share from roughly a third to roughly a quarter each, assuming even allocation; because REQ carries no data and DAT carries all of it, Phase 0's arbitration should weight the four channels' shares unevenly (DAT and SNP heavier than REQ and RSP) rather than dividing bandwidth uniformly; Phase 0 therefore requires priority-weighted bandwidth allocation at the chip bridge DAT and SNP given a larger fixed share than REQ and RSP as a specified arbitration policy, not an optional post-implementation tuning pass. An even four-way split is explicitly rejected as a default, since it regresses off-chip throughput relative to the native three-NoC design on any workload with a typical read/write data-to-control ratio.

8. Explicitly Out of Scope For Phase 0

Phase 0 is bounded on purpose: it converts the protocol layer and closes the conflict-handling gap, and stops there. The following are real, valuable extensions, deferred to later, separately reviewable phases so Phase 0 stays mergeable on its own.

Phase 1: Richer Snoop Types, Exclusive Atomics, And Block-Store / Prefetch Reactivation

CHI defines a family of snoop types (for example SnpOnce, SnpUnique, SnpCleanInvalid) with distinct data-return and ownership-transfer rules, more expressive than OpenPiton's current Inv/FwdRd/FwdExRd set. Phase 0 maps native forwards onto a functionally equivalent, reduced CHI snoop subset; Phase 1 would expand this to the full snoop-type set. Phase 1 would also replace the L1.5's bespoke two-phase CAS/SWP/LOADSTUB handling (specification Section 4.5.5) with CHI's standardized exclusive-access monitor sequence.

Phase 1 is also where the block-store and prefetch reactivation belong. Both paths exist in OpenPiton's packet format today but are functionally disabled: prefetch requests are treated as no-ops at the L1.5 (specification Section 4.5.2), and block-store requests are decomposed into a sequence of regular per-word stores before they ever reach the L2 (Section 4.5.3), which means the L2 and NoC1 never see the operation as a single unit worth optimizing. Section 4 lists both as identified gaps; they were left off Phase 0's roadmap deliberately, since re-enabling them is an optimization on top of a working protocol rather than a prerequisite for the protocol conversion itself, but they belong on the roadmap explicitly rather than only in the gaps list, since CHI is what makes reactivating them worthwhile:

- Block-store → CHI WriteUnique with a multi-beat DAT burst. Instead of decomposing a block-store into N separate ReqExRd/store transactions each carrying its own MSHR entry and REQ-channel round trip, Phase 1 issues one WriteUnique (or WriteBack, if the line is already owned) whose data moves as a single multi-beat DAT burst. This is a direct win from the channel separation. Section 6.2 introduces that native OpenPiton has no channel that can carry a request and its trailing data burst as a distinguishable unit the way CHI's DAT channel can.
- Prefetch → CHI ReadNoSnp / ReadNotSharedDirty as a non-blocking hint. Rather than a no-op, a prefetch becomes a CHI read that allocates into the RN-F's cache on completion without stalling the issuing thread's pipeline; the RN-F simply does not block waiting on it the way it would for a demand load. This requires the RN-F MSHR (already being extended for retry/credit tracking, Section 7) to support a lower-priority, droppable class of outstanding transaction, so a full MSHR sheds prefetches before it stalls demand traffic.



Proprietary and Confidential

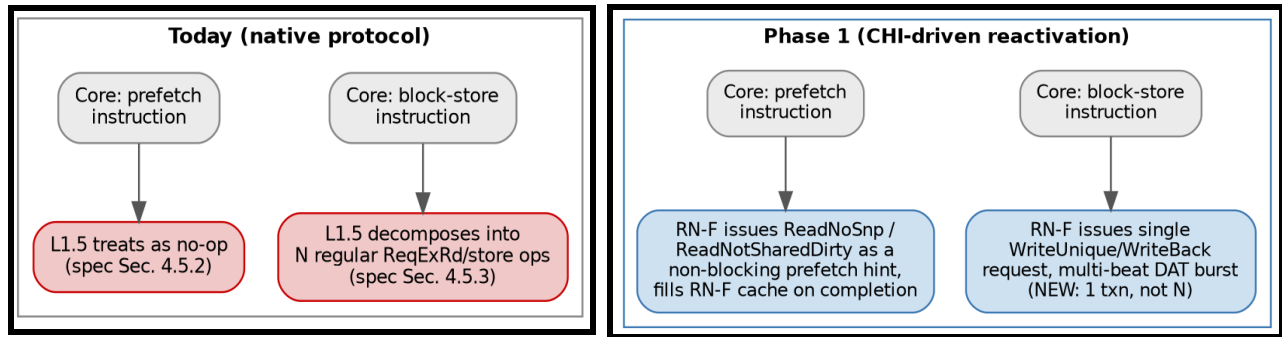


Figure 7: Phase 1 reactivation of the currently-disabled block-store and prefetch paths, contrasted with their present no-op/decomposed behavior

Phase 2: Directory Scaling And CDR Integration

Widening the directory beyond the current 64-sharer bit-vector (specification Section 5.1) using CHI's node-ID space, and reconciling Coherence Domain Restriction currently marked experimental (specification Section 2.3.3, [6]) with a CHI-based home node.

Phase 3: DVM Broadcast Operations And Multi-Chip CHI Export

Replacing OpenPiton's purpose-built inter-processor-interrupt mechanism (specification Section 4.7) with CHI's Distributed Virtual Memory (DVM) messages for system-wide operations, and extending the chip-to-chip coherence extension (specification Section 2.6.1) to carry CHI natively between chips rather than OpenPiton's native inter-chip routing protocol. This is also where the transition-period IPI sideband flagged in Section 7.1 is finally retired.

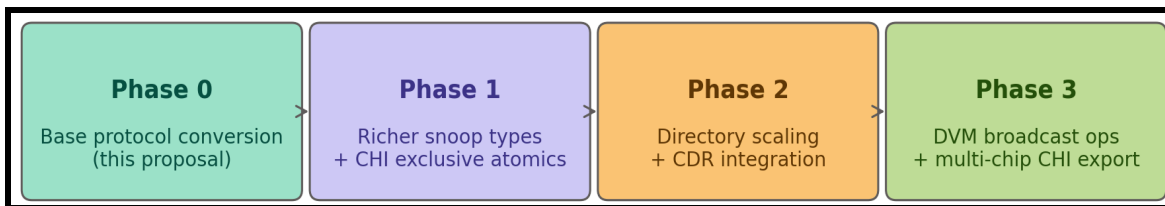


Figure 8: Phased roadmap.

Each phase is independently mergeable and separately reviewable; this document specifies Phase 0 only

9. Broader Impact On The OpenPiton Project

A protocol-layer change of this kind touches more than the coherence RTL itself. This section is an honest accounting of what else moves.

- Regression suite: OpenPiton ships roughly 9,000 tests [1, Sec. 1]. Every test that exercises coherence traffic needs to run under both native and CHI modes during the transition, which argues for the build-time toggle mentioned in Section 7 rather than a hard cutover.



Proprietary and Confidential

- Specification and documentation: Chapters 3 (coherence protocol), 4 (L1.5), 5 (L2), and 6 (on-chip network) of the current specification describe the native protocol in detail and would need a parallel CHI-mode chapter set, not a rewrite-in-place, so the native protocol remains documented for existing users.
- Core integrations: OpenPiton's CCX/L1.5 boundary is already the substitution point used by other cores integrated with the platform, including CVA6 and BlackParrot via BYOC [4]. Any core integrated at that boundary is affected by the RN-F transducer change in Section 7 and needs re-validation against it this is the largest indirect blast radius of the proposal.
- Chip bridge and inter-chip routing: extending from three to four multiplexed channels (Section 7) has a direct, bounded impact on the off-chip link's bandwidth accounting, described in the specification's Section 2.5.2 and quantified further in Section 7.1.
- Build/configuration system: a new configuration axis (protocol mode) needs to be threaded through the existing single-configuration-file build system (specification Section 2.2) alongside the existing core-count, cache-size, and topology options. Because native and CHI logic are kept in separate module instances (Section 7) rather than merged, this configuration axis affects module selection at build time, not per-module control logic, limiting the timing-closure risk to each mode's own critical path rather than compounding both into a single path.

None of these are blockers; they are the reason Phase 0 is scoped narrowly and kept backward-compatible rather than replacing the native protocol outright.

10. Anticipated Reviewer Questions

Q1. Why CHI and not AMBA ACE?

Answer1. ACE is AXI's coherency extension, built for smaller coherent clusters; CHI is ACE's successor specifically designed for larger-scale, higher-core-count coherent systems with a layered protocol/network/link architecture a closer match to OpenPiton's manycore, distributed-directory design point than ACE's more tightly coupled model.

Q2. Does this change break TSO?

Answer2. Not if Section 6.4's ordering-point mapping is implemented as specified. CHI does not provide TSO for free the way OpenPiton's native NoC ordering does today, so Phase 0 must explicitly reconstruct it from CHI's order-point and per-channel ordering primitives, argued property-by-property in Section 6.4.2; this is called out as the one genuinely novel design element of Phase 0, not an incidental detail.

Q3. Why keep the native protocol as an option instead of replacing it?

Answer3. Because the existing 9,000-test regression suite, existing core integrations, and existing published research built on OpenPiton all depend on the native protocol continuing to work. A build-time toggle protects all of that while the CHI *path matures*.

Q4. How does this relate to the BYOC / BlackParrot-BedRock work?

Answer4. BYOC already proves the CCX/L1.5 boundary is a valid substitution point, by using it to attach different cores and coherence systems [4]. This proposal targets the same boundary from the opposite direction, not to plug in a different core, but to make the protocol on the coherence side of that boundary a standard one. The two efforts are complementary, not competing.



Proprietary and Confidential

Q5. What happens to Coherence Domain Restriction (CDR)?

Answer5. CDR is explicitly marked experimental in the current specification [1, Sec. 2.3.3] and is left untouched in Phase 0. Reconciling it with a CHI-based home node is scoped to Phase 2 (Section 8), once the base conversion is stable.

Q6. Why weren't block-store and prefetch reactivated in Phase 0?

Answer6. Because they are optimizations layered on top of a working protocol, not prerequisites for the protocol conversion. Phase 0's mandate (Section 6) is behavioral equivalence to today's system, and today's system already works correctly with prefetch as a no-op and block-store decomposed. Section 8's Phase 1 reactivates both once the CHI channel separation that makes them worthwhile (Section 6.2) is in place and stable.

Q7. What is the impact on area and timing?

Answer7. Area and timing closure are downstream, process- and configuration-dependent concerns and remain out of scope for final numbers in this document. However, one architectural cost is not deferred: Section 6.3.2 commits to four physical CHI lanes replacing OpenPiton's three physical NoCs, which increases router crossbar port count and mesh wire-track count at every hop this is not reuse of existing NoC1–NoC3 wiring, but an addition of a fourth track throughout `dynamic_node_top.v` and the tile mesh. The RTL change inventory (Section 7) is scoped so that once a target configuration (core count, cache sizes, process node) is chosen, this specific incremental cost three physical lanes to four can be estimated directly, rather than assumed to be negligible.

11. References

- [1]** Wentzlaff Parallel Research Group, Princeton University. *OpenPiton Microarchitecture Specification, Rev. 1.0, 2016.*
- [2]** J. Balkind, M. McKeown, Y. Fu, T. Nguyen, Y. Zhou, A. Lavrov, M. Shahrad, A. Fuchs, S. Payne, X. Liang, M. Matl, D. Wentzlaff. "OpenPiton: An Open Source Manycore Research Framework." *ASPLOS, 2016.*
- [3]** Y. Shan. "Practical Cache Coherence." *Personal technical notes, lastweek.io observes that the OpenPiton specification does not document coherence transaction conflict handling, in contrast to the AMBA CHI specification's dedicated interconnect protocol flow chapter.*
- [4]** "The Open-Source BlackParrot-BedRock Cache Coherence System" (BYOC), *arXiv:2505.00962.*
- [5]** "EPAC: The Last Dance," *arXiv:2604.12715, a tiled manycore uncore built natively on AMBA CHI with a distributed, CHI home-node-coupled L2.*
- [6]** Y. Fu, T. M. Nguyen, D. Wentzlaff. "Coherence Domain Restriction on Large Scale Systems." *MICRO-48, 2015.*
- [7]** ARM Ltd. *AMBA 5 CHI Architecture Specification. arm.com.*



Proprietary and Confidential